

with various hardware devices. To manage all these processes, the kernel has privileged access and the code that executes in this space has to be well written and verified to maintain the overall performance of the system.

On the other hand, the user space has the applications written by the users which will utilise the system resources through system calls. These system calls request the kernel to perform an action on behalf of the user application. The user applications include application tasks, libraries, scripts, etc, which execute in the user space.

Debugging your embedded application code

The application code resides in user space. These applications can be debugged by using various tools provided by Linux, GDB (GNU Project Debugger) being one of the basic debuggers in the Linux environment. GDB provides a command line interface for debugging the applications. There are various front end tools built on it.

Compiling your code with debug symbols

To debug a piece of code, debug symbols have to be included in the executable of the code. These debug symbols are usually stripped off during the compilation stages in order to reduce the footprint of the final executable. To include these debug symbols in the binary executable so that it enables debugging, compile the code with the `-g` option. Let's consider the small code snippet in Figure 1.

Compiling the code below gives an executable without debugging information as in Figure 2.

```
gcc snip.c -o <executable name >>
```

Compiling the code as shown below with the `-g` option gives an executable with debugging information, as in Figure 3.

```
gcc -g snip.c -o <executable name >>
```

The presence of debug information in an executable can be verified by having an object dump (*objdump*) of the executable.

```
Example: objdump -h <executable name>
```

Now, to debug the code, GDB should be started on the terminal.

```
gdb << executable name>>
```

GDB loads the symbols and shows the *gdb* prompt.

```
(gdb)
```

GDB provides a list of options to view the content of files, put breakpoints and execute the code. These options can read

```
/**File Name: snip.c**/

main()
{
    int number, name;

    number=8190;

    name="example";

    printf("%d\n",number);

    printf("%s\n",name);
}
```

Figure 1: Code snippet

```
//gives executable without debugging information
gcc snip.c -o <executable name >>

//object dump of the binary executable.
-dis 00000040 00000040 00000040 00000040 2**2 CONTENTS, ALLOC, LOAD, READONLY, CODE
+text 00000010 00000010 00000010 00000010 2**4 CONTENTS, ALLOC, LOAD, READONLY, CODE
+fini 00000010 00000010 00000010 00000010 2**2 CONTENTS, ALLOC, LOAD, READONLY, CODE
+data 00000000 00000000 00000000 00000000 2**2 CONTENTS, ALLOC, LOAD, READONLY, DATA
+ro_data 00000000 00000000 00000000 00000000 2**2 CONTENTS, ALLOC, LOAD, READONLY, DATA
+ctors 00000000 00000000 00000000 00000000 2**2 CONTENTS, ALLOC, LOAD, DATA
+dtors 00000000 00000000 00000000 00000000 2**2 CONTENTS, ALLOC, LOAD, DATA
+gnu 00000000 00000000 00000000 00000000 2**2 CONTENTS, ALLOC, LOAD, DATA
+dynamic 00000000 00000000 00000000 00000000 2**2 CONTENTS, ALLOC, LOAD, DATA
+got 00000000 00000000 00000000 00000000 2**2 CONTENTS, ALLOC, LOAD, DATA
+got.plt 00000000 00000000 00000000 00000000 2**2 CONTENTS, ALLOC, LOAD, DATA
+data 00000000 00000000 00000000 00000000 2**2 CONTENTS, ALLOC, LOAD, DATA
+bss 00000000 00000000 00000000 00000000 2**2 ALLOC
+comment 00000000 00000000 00000000 00000000 2**0
```

Figure 2: Without debug information

```
//gives executable with debugging information
gcc -g snip.c -o <executable name >>

//Check out the inclusion of debug information in the object dump of the
binary executable.
.bss 00000000 00000000 00000000 00000000 2**2 ALLOC
+comment 00000000 00000000 00000000 00000000 2**0 CONTENTS, READONLY
+debug_ranges 00000000 00000000 00000000 00000000 2**0 CONTENTS, READONLY, DEBUGRANGES
+debug_got_names 00000000 00000000 00000000 00000000 2**0 CONTENTS, READONLY, DEBUGRANGES
+debug_info 00000000 00000000 00000000 00000000 2**0 CONTENTS, READONLY, DEBUGRANGES
+debug_abbrev 00000000 00000000 00000000 00000000 2**0 CONTENTS, READONLY, DEBUGRANGES
+debug_line 00000000 00000000 00000000 00000000 2**0 CONTENTS, READONLY, DEBUGRANGES
+debug_frame 00000000 00000000 00000000 00000000 2**0 CONTENTS, READONLY, DEBUGRANGES
+debug_etc 00000000 00000000 00000000 00000000 2**0 CONTENTS, READONLY, DEBUGRANGES
+debug_loc 00000000 00000000 00000000 00000000 2**0 CONTENTS, READONLY, DEBUGRANGES
```

Figure 3: With debug information

by the help command inside the *gdb* prompt as in Figure 4.

Remote debugging using GDB for an embedded environment

In remote debugging, the application executes on the embedded target and the same application is debugged from the host PC. To do this, compile the '*gdbserver*' for the embedded target environment.